

CS4780 - Lab 1: R Introduction

Contents

Background	2
Features of R	2
R Studio	2
Getting R and R Studio	2
Downloading and installing on your own machine	2
Getting help	3
Online resources	3
Working in RStudio	3
Setting yourself up: The R workspace	4
Important Note to Windows Users	5
Workspace data	5
Exercise 1	5
R language constructs	5
Comments	5
Data structures	5
Data frames and datasets	6
Importing data	7
Exercise 2	7
Manipulating variables	7
Exercise 3	8
Subsets	8
Exercise 4	8
Manipulating observations	8
Exercise 5	9
Using packages	9
Installing packages	9
Loading packages	9
Reading the package manual and vignette	10
Saving R history	10

Submitting your Lab assignment

For this lab, you do not need to create a separate .R file, but just save the history of your R Studio session. At the end of the lab, please submit the command history of your R Studio session via the corresponding link on Blackboard. For more information, see section “[Saving R history](#)” at the end of the lab.

Submit the history either as an R Script (.Rhistory) or as plain text file (.txt). *No other formats will be considered.*

Background

R (<https://www.r-project.org/>) is a system for programming language and environment geared towards statistical analysis and data visualisation. Historically, R is a re-implementation of the S programming created by John Chambers in 1976 at Bell Labs.

R is open source under the terms of the Free Software Foundation’s GNU General Public License.

Over the past few years, the R ecosystem (composed of the basic language, community-contributed packages, specialised IDEs, etc) has become one of the top resources (together with Python) for statistical analysis and data science, both in the scientific community and in data-intensive industries such as Google, Uber and Microsoft (<https://www.northeastern.edu/levelblog/2017/05/31/big-companies-using-r-data-analysis/>).

Features of R

Some of the features of R include:

- effective data handling and storage
- operators for calculations on matrices
- graphical facilities for data analysis and display
- can be extended via packages

One of the strengths of R is the many extension, called “packages” that are available. These extensions range from web scraping to plotting to machine learning to advanced statistical methods. R packages are centrally maintained via the Comprehensive R Archive Network (<https://cran.r-project.org/>). As of October 2019, the CRAN package repository features 15104 packages.

R Studio

RStudio is a free, open-source integrated development environment (IDE) for R. It provides users with tools for package development, graphical exploration, report generation, interactive app development and other extensions to the basic R language. The Rstudio team also actively develops some of the most widely used R packages for data science, such as dplyr (for data preprocessing), ggplot2 (for graphics), rmarkdown (for report generation) and several others.

Getting R and R Studio

You should be able to launch R Studio from the AstonApps page. A glitch I have noticed is that you need to first launch R before you can launch R Studio.

Downloading and installing on your own machine

Instructions to download R:

1. Go to <https://cran.r-project.org/>.

2. Click the link at the top for your operating system.
3. For Windows, click ‘base’ and then click the download link at the top of the page.
4. For Mac, under the ‘Files’ section on the next page, click on the first ‘.pkg’ link.

Instructions to download RStudio:

1. Go to <https://www.rstudio.com/>.
2. Click on the “Download” link below the RStudio icon.
3. Click on the “Download” link for the RStudio Desktop, Open Source License.
4. Click on the link under “Installers for Supported Platforms” for your operating system.

Getting help

Every function in R comes with documentation. To get the documentation for a function, you can either type a `?` before the function name or type `help([FUNCTION NAME])`. For example, let’s look at the help file for the `read.csv(...)` function.

Note: `[...]` means your argument that you pass to the function.

```
# Pulls up help file for the read.csv command
help(read.csv)
```

You should see the documentation, titled ‘Data Input,’ pop up in the bottom right corner of your RStudio window.

The ‘Usage’ section, next, is important. In this case, the help file is showing the usage of several commands, including `read.table(...)` and `read.delim(...)`, since they all have similar usage. Look at the usage for `read.csv(...)`. You should see this:

```
read.csv(file, header = TRUE, sep = ",", quote = "\"",
         dec = ".", fill = TRUE, comment.char = "", ...)
```

Note that one of the most useful parts is often at the bottom: ‘Examples’.

Online resources

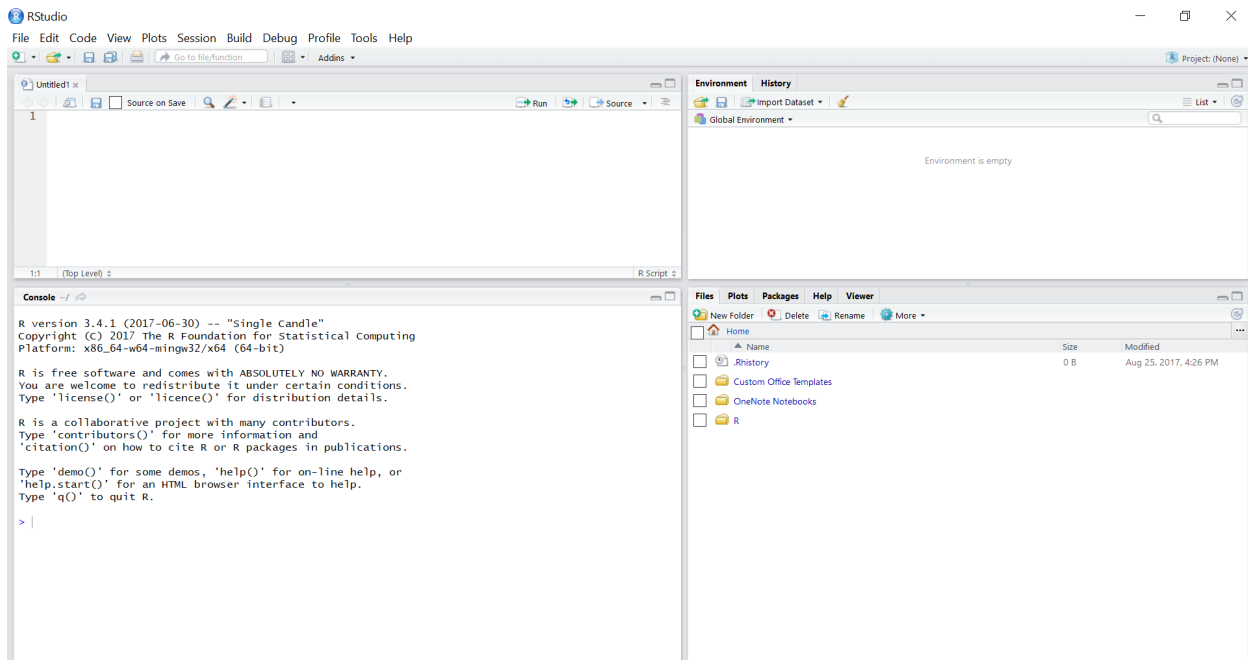
- [Quick R website](#)
- [RProgramming.net](#)
- [UCLA Institute for Digital Research and Education](#)
- *O’Reilly R Cookbook*
- *Data Analysis Using Regression and Multilevel/Hierarchical Models* by Andrew Gelman and Jennifer Hill

An important skill is the ability to think of something you want to do with the data, understand the language to search for a solution, and implement that solution.

Google is your friend. Most things you will want to do, someone else has done before. And most of the time, a solution has been posted online. The R mailing list and [Stack Overflow](#) are a good resource to post questions and look for answers.

Working in RStudio

When you open RStudio, go to **File > New File > R Script** Overall, your screen will look something like this.



Broadly, the four areas you see on your screen are as follows:

- Top-left: Script editor, where you can write code and run code from a script
- Bottom-left: Console, where you can run R commands one-by-one and see results
- Top-right: Environment, which will list your R objects (will be explained)
- Bottom-right: Where you will see plots you create, help files, etc.

This tutorial will run through a series of commands. In general, you can try any commands in the console region to see if they work. *For this tutorial, instead of typing directly into the console, type the commands into the R script window. To run the command after you've typed it, highlight the code, and then press the "Run" button in the top right of the script area.* The purpose of this is to let you practice creating a save-able script that you can return to in the future.

Setting yourself up: The R workspace

The workspace is your current R working environment and includes any work and data for your current project. At the end of an R session, the user can save an image of the current workspace that is automatically reloaded the next time R is started. Commands are entered interactively at the R user prompt. **Up** and **down arrow keys** scroll through your command history.

You will probably want to keep different projects in different physical directories. Here are some standard commands for managing your workspace.

You can set the working directory by using the `setwd([...])` command. So, run the `setwd([...])` command as shown below.

If you are using a Mac, it will look something like this:

```
setwd("~/Desktop/R Introduction")
```

If you are using Windows, it will look something like this:

```
setwd("C:/Users/Ulysses/Desktop/R Introduction")
```

You can use the `getwd()` command to print the current working directory. To list the objects in the current workspace use the `ls()` command.

Working with your previous commands

```
history() # display last 25 commands
history(max.show=Inf) # display all previous commands
```

Sometimes it is useful to save and load the history of your commands. You can save your command history with `savehistory(file="[myfile]")` and recall your command history with `loadhistory(file="[myfile]")`. The default file is named 'Rhistory'.

Important Note to Windows Users

R gets confused if you use a path in your code like: `c:\mydocuments\myfile.txt`. This is because R sees "`\`" as an escape character. Instead, use: `c:\\my documents\\myfile.txt` or `c:/mydocuments/myfile.txt`

Workspace data

Whenever you create a data structure it automatically becomes part of the workspace data. However, this workspace data is not automatically saved to disk. You can save the workspace to the file 'RData' in the current working directory using `save.image([...])`. To load a workspace into the current session, use `load("myfile.RData")` (if you don't specify the path, the current working directory is assumed). Note that when you quit R, you will be prompted to save the workspace. By default, R loads the previous workspace when you start the application.

Exercise 1

Find out what your current working directory is.

R language constructs

Comments

Comments in R are denoted with the `#` symbol. R does not support multi-line comments.

Data structures

Much of the logic of working with R revolves around information and ways to store that information in 'structures'

Scalar variables

In R values are assigned to variables using the `<-` operator

```
a <- 2
```

Note that the assignment operator does not return the assigned value (as e.g. does Matlab). To inspect the variable, simply type its name `a`.

If you wish to delete a variable from your workspace, use the `rm([...])` command.

```
# Removes the myvar variable
myvar = 1/10
rm(myvar)
```

Notice that the 'myvar' object is now gone from your environment window.

Lists

Variables can hold multiple values at once. Using the `c(...)` you can create a list of values:

```
a <- c(5, 4, 1, 3, 7)
a
```

```
## [1] 5 4 1 3 7
```

Notice that even though we already had an `a` variable from before, we can overwrite the information easily by assigning new information to it.

You can reference different elements in the list by referring to the index of the element. Note that R starts counting indices at **1** (and not 0).

```
a[5]
```

```
## [1] 7
```

Matrices

As expected, matrices are 2-dimensional lists. Consider the following matrix.

7	12	jackfruit
Madonna	0	What?

This is a 2x3 matrix with two rows and three columns. You could load it into R as follows.

```
brian <- matrix(c("7", "Madonna", "12", "0", "jackfruit", "What?"),
               nrow=2, ncol=3)
brian
```

```
##      [,1]      [,2] [,3]
## [1,] "7"      "12"  "jackfruit"
## [2,] "Madonna" "0"   "What?"
```

Like lists, you can refer to matrix elements by index. You use the same `[]` notation, but now you put two numbers, separated by a comma, to indicate the row and column, respectively.

```
brian[2,1]
```

```
## [1] "Madonna"
```

If you just want to get information from an entire row, leave out the column number

```
brian[2,]
```

```
## [1] "Madonna" "0"      "What?"
```

And vice versa to get information from an entire column.

```
brian[,1]
```

```
## [1] "7"      "Madonna"
```

Data frames and datasets

‘Data Frames’ are the most used data structure in R. Data Frames are similar to matrices, but you can also refer to columns by name.

For most things we do in R, we will first need to get data into the workspace. This can be done by opening an existing R workspace that already contains data, or by importing new data.

Note that the standard R package ‘R Datasets’ provides quite a few data sets that you use to get started. For a complete list, use `library(help = "datasets")` or look here: <https://stat.ethz.ch/R-manual/R-devel/library/datasets/html/00Index.html>.

Importing data

One of the easiest ways to get data into R is to import the data from a file in the Comma Separated Values (CSV) format. We will do this using the `read.csv(...)` command. In your current working directory, create a folder named “data”, download the data file from the ‘data’ folder on BlackBoard or from [this link](#), and store it in the “data” folder.

```
# Reads in the data into the 'mycars' data structure
mycars <- read.csv("data/mycars.csv")
```

Note that this CSV file contains the same data as the built-in ‘mtcars’ data set. So if you have a problem importing the file, you can get the data via `mycars <- mtcars`.

In your environment window, you can see that we have now loaded the data into a data structure called `mycars`. You can view the dataset using the `View()` command or by just clicking on the name of the data structure in your environment window.

The data should look something like this (the table below is truncated):

make	mpg	cyl	displacement	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

Exercise 2

What is the command to save the dataframe `mycars` to a CSV file?

Manipulating variables

Before we get into the data analysis, let’s cover a couple of basic data manipulation things.

Because you are able to have different datasets open at once in different data structures, variable references need to be associated with the specific data structure using the `$` symbol. For example, if I wanted to get a summary of the `mpg` variable in `mycars`, I would type...

```
# Gets summary statistics for the cst28_engl variable
summary(mycars$mpg)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  10.40   15.43   19.20   20.09   22.80   33.90
```

To add a variable, just assign something to it.

```
# Calculate liters per 100 KM from gallon per mile
mycars$l100k <- 100/((mycars$mpg * 1.609)/4.546)
```

make	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb	lp100k
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4	13.45408
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4	13.45408
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1	12.39192
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1	13.20260
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2	15.10886
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1	15.60971

If you view the `mycars` dataset, you can now see the `lp100k` variable tagged onto the end.

To delete a variable, we simply assign `NULL` to it.

```
# Delete the lp100k variable
mycars$lp100k <- NULL
```

Exercise 3

Add a new variable that represents gallons per mile.

Subsets

Sometimes, you will want to create a subset of a data set, i.e. only some of the variables. For example, say you only wanted the `make` and `hp` variables. You can do this by creating a list of the variables you want using the `c(...)` command and then ‘subsetting’ by name, as shown below.

```
# List of variables
vars <- c("make", "hp")
# Get subset
mycarssubset <- mycars[vars]
```

You can click on the `mycarssubset` data structure in the environment window to make sure that it worked. Note that I could have done the above in one line, as shown below.

```
# Get subset
mycarssubset <- mycars[c("make", "hp")]
```

Exercise 4

Create a new subset of `mycars` with the `make`, `mpg`, and `cyl` variables

Manipulating observations

To refer to specific observations, you can do it by row number, but that’s not how we typically do it in data analysis. For example, the following code prints the information from the fourth row, as per the code we learned earlier about matrices.

```
mycars[4,]

##           make  mpg  cyl  disp  hp  drat    wt  qsec vs  am  gear  carb
## 4 Hornet 4 Drive 21.4   6  258  110 3.08 3.215 19.44 1  0    3    1
```

Typically, we use some condition. Let’s say we wanted to refer to only those cars that have more than 6 cylinders.


```
# Creates newmycars data structure, subset of mycars data structure with car with more than 6 cylinders
newmycars <- mycars[mycars$cyl>6,]
newmycars
```

```
##           make  mpg  cyl  disp  hp drat   wt  qsec vs  am  gear  carb
## 5   Hornet Sportabout 18.7    8 360.0 175 3.15 3.440 17.02 0  0    3    2
## 7           Duster 360 14.3    8 360.0 245 3.21 3.570 15.84 0  0    3    4
## 12          Merc 450SE 16.4    8 275.8 180 3.07 4.070 17.40 0  0    3    3
## 13          Merc 450SL 17.3    8 275.8 180 3.07 3.730 17.60 0  0    3    3
## 14          Merc 450SLC 15.2    8 275.8 180 3.07 3.780 18.00 0  0    3    3
## 15  Cadillac Fleetwood 10.4    8 472.0 205 2.93 5.250 17.98 0  0    3    4
## 16 Lincoln Continental 10.4    8 460.0 215 3.00 5.424 17.82 0  0    3    4
## 17  Chrysler Imperial 14.7    8 440.0 230 3.23 5.345 17.42 0  0    3    4
## 22   Dodge Challenger 15.5    8 318.0 150 2.76 3.520 16.87 0  0    3    2
## 23         AMC Javelin 15.2    8 304.0 150 3.15 3.435 17.30 0  0    3    2
## 24         Camaro Z28 13.3    8 350.0 245 3.73 3.840 15.41 0  0    3    4
## 25   Pontiac Firebird 19.2    8 400.0 175 3.08 3.845 17.05 0  0    3    2
## 29     Ford Pantera L 15.8    8 351.0 264 4.22 3.170 14.50 0  1    5    4
## 31     Maserati Bora 15.0    8 301.0 335 3.54 3.570 14.60 0  1    5    8
```

This single command did quite a few things at once, let's unpack it a bit. The first thing is to look at `mycars$cyl>6`. What this command actually does, is return a vector of the same length as 'mycars', where all the row for which the condition `cyl>6` is true is set to `TRUE` and all the others to `FALSE`:

```
mycars$cyl>6
```

```
## [1] FALSE FALSE FALSE FALSE  TRUE FALSE  TRUE FALSE FALSE FALSE FALSE  TRUE
## [13]  TRUE  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE
## [25]  TRUE FALSE FALSE FALSE  TRUE FALSE  TRUE FALSE
```

What the statement `mycars[mycars$cyl>6,]` actually does is get from 'mycars' all the rows where `mycars$cyl>6` was `TRUE`.

Exercise 5

Get all the cars with more than 20 and less than 30 miles per gallon.

Using packages

Installing packages

To install a package use the `install.packages(...)` function. This simple command downloads the package from a specified repository (by default, CRAN) and installs it on your machine:

```
install.packages("fortunes")
```

Note that the argument to `install.packages(...)` is a character string. In other words, remember the quotes around the package name!

In RStudio, you find a menu command to do the same thing: * In RStudio, choose **Tool->Install packages...**

Loading packages

To load a package, you use the `library(...)` or `require(...)` function. These functions are identical in their effects, but they differ in the return value: * `library(...)`: Invisibly returns a list of packages

that are attached, or stops with an error if the package is not on your machine. * `require(...)`: Returns TRUE if the package was successfully attached and FALSE if not.

The R documentation suggests that `library(...)` is the preferred way of loading packages in scripts, while `require(...)` is preferred inside functions and packages.

So, after installing the package `fortunes` you load it like this:

```
library("fortunes")
```

Note that you don't have to quote the name of the package in the argument of `library(...)`, but it is good practice to always quote the package name.

Although it is possible to unload a package within an R session by using the `detach(...)` function, in practice it usually is much easier to simply restart your R session.

Reading the package manual and vignette

The package manual is a collection of all functions and other package documentation. You can access the manual in two ways. The first way is to use the `help` argument to the `library(...)` function:

```
library(help = "fortunes")
```

The second way is to find the manual on the package website. If you point your browser window to the [CRAN page for the fortunes package](#), you'll notice a [link to the manual](#) toward the bottom of the page.

Whichever approach you choose, the result is a PDF document containing the package manual.

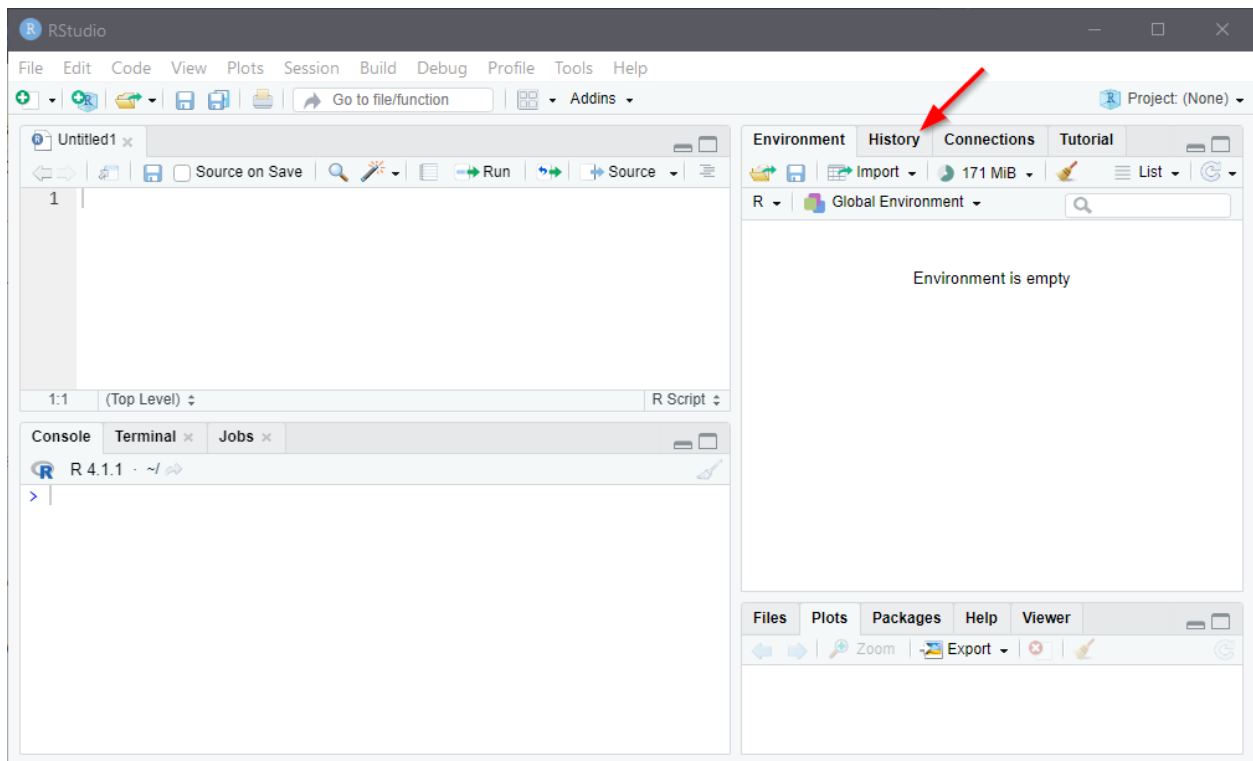
Some package authors also write one or more *vignettes*, documents that illustrate how to use the package. A vignette typically shows some examples of how to use the functions and how to get started. The key thing is that a vignette illustrates how to use the package with R code and output, just like this book.

To read the vignette for the `fortunes` package, try the following:

```
vignette("fortunes")
```

Saving R history

To save the command history of your R Studio session, switch to the **History** tab.



Next, save the history:

